Predicting Feynman Periods with Graph Neural Networks

Project Milestone

Members: Ethan Katz, Evelyn Song, Colin McKhann
Project Type: Real-world applications of GNNs
Code: https://github.com/EthanKatz/cs224w-feynman-periods

1 Physical Background

Why the β function matters. In quantum field theory the strength of an interaction is not fixed: the coupling g depends on the energy scale μ at which it is probed. The β function describes this running,

$$\beta(g) = \frac{dg}{d\log\mu}.$$

It is what connects a measurement at one energy to a prediction at another. Its zeros are fixed points that describe systems at continuous phase transitions, and its slopes there give universal *critical exponents*, such as those of a magnet at its Curie point or a fluid at its liquid–gas critical point. ϕ^4-theory is the standard model for these phenomena, and its β function yields some of the most precise predictions in field theory (Kompaniets and Panzer, 2017, Wilson and Kogut, 1974). In practice β is computed as a perturbative series,

$$\beta(g) = \sum_{L\geq 1} \beta_L \, g^{L+1},$$

whose coefficients grow factorially with the loop number L. Extracting accurate numbers therefore requires resummation, which in turn needs knowledge of the coefficients at large L.

How periods enter. The L-loop coefficient β_L is a sum over Feynman graphs with L loops. Each *primitive* (subdivergence-free) graph G contributes a single number, its *period*

$$\mathcal{P}(G) = \int_{\alpha_e>0} \prod_{e\in E} d\alpha_e \, \frac{\delta\left(1-\sum_{e\in E}\alpha_e\right)}{\psi_G(\alpha)^2}, \qquad \psi_G(\alpha) = \sum_{T \text{ spanning tree}} \prod_{e\notin T} \alpha_e,$$

weighted by its symmetry factor, so that up to combinatorial normalization

$$\beta_L^{\text{prim}} \;\propto \sum_{\text{primitive } G,\ L \text{ loops}} \frac{\mathcal{P}(G)}{|\mathrm{Aut}(G)|}.$$

Periods do not depend on the renormalization scheme, which makes them clean, well-defined targets. Non-primitive graphs contain nested divergences and require further subtractions; we do not treat them.

How amplitudes fit in. An amplitude is the quantum probability amplitude of a scattering process, obtained by summing Feynman graphs. Schematically, after renormalization a primitive graph contributes

$$\mathcal{A}_G(p) \;=\; \mathcal{P}(G)\,\log\!\left(\frac{p^2}{\mu^2}\right) \;+\; c_G, \qquad \mathcal{A}(p) \;=\; \sum_G \frac{g^{L+1}}{|\mathrm{Aut}(G)|}\,\mathcal{A}_G(p),$$

where p is the scattering momentum and c_G a scheme-dependent finite part. The period is thus the coefficient of the logarithm: it sets how strongly the graph's contribution changes with energy. The β function collects exactly these coefficients, and the renormalization-group (Callan–Symanzik) equation then uses β to fix the energy dependence of the full amplitude. Predicting periods therefore does *not* yield complete amplitudes (it misses the finite parts c_G and the non-primitive graphs), but it targets their universal, expensive core.

Implication for evaluation. Physical applications need *sums* of periods over many graphs. Random per-graph errors largely cancel in such sums, whereas a systematic bias does not. This makes errors in the overall level of the predictions, as arise when extrapolating to a new loop order (Section 5), particularly important.

2 Problem and Motivation

The number of primitive graphs grows factorially with the loop number L, and each period is an expensive numerical integral, so a cheap and accurate predictor of $\mathcal{P}(G)$ from the graph alone would directly reduce the cost of high-order QFT calculations. Balduf and Shaban (2024a) showed that structural invariants (notably the Hepp bound, Martin invariants and cycle/cut counts) predict periods well, but the strongest of these are themselves costly to compute, and their GNNs, which started from arbitrary vertex labels, never beat linear regression. Our project asks whether message-passing GNNs, given *cheap structural node features*, can learn periods from graph structure alone.

Milestone summary. We built a complete, reproducible pipeline (download → parse → PyG `Data` → grouped splits → training/evaluation of GCN, GraphSAGE and GIN → non-GNN baselines), and trained 54 GNN models (3 architectures × 9 node-feature sets × 2 seeds) on all 92,680 primitive graphs with $3 \le L \le 12$, plus 18 runs of a loop-order extrapolation experiment. Together these form the ablation of our proposed novelty: structural node features versus constant or arbitrary ones. Headline results: (i) as predicted by a 1-WL argument, GNNs with constant features are no better than linear regression on L (42% vs. 46% mean relative error); (ii) with per-node cycle participation, Laplacian and random-walk encodings, GNNs reach **0.97%** mean relative error, 7.5× better than ridge regression on the same cycle counts; (iii) trained only on $L \le 11$, the best GNN predicts unseen 12-loop periods with 4.3% error, once it is asked to predict the residual of a linear fit in L. The expensive Hepp bound remains a stronger single predictor (0.6%).

3 Dataset and Processing

Source. We use the public dataset of Balduf and Shaban (2024b) (Borealis Dataverse, CC-BY 4.0), which accompanies Balduf and Shaban (2024a) and Balduf (2023). File `periods_L.csv` lists one graph per line, with its numerical period, its uncertainty, the Hepp bound and further invariants. The dataset is *complete* up to $L = 13$ and a non-uniform sample for $14 \le L \le 18$.

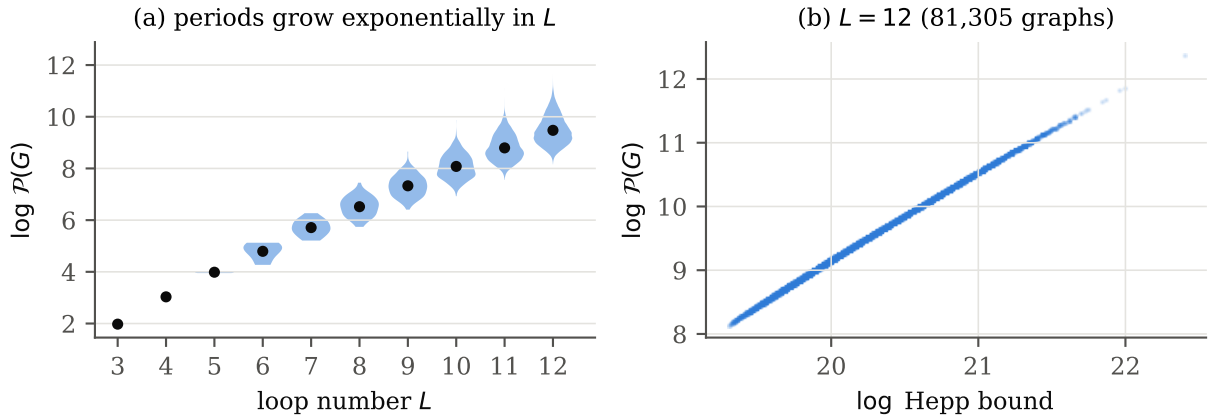


Figure 1: (a) Distribution of $\log \mathcal{P}$ per loop order: the loop number explains the mean, but within one L periods still span more than an order of magnitude (a factor of $\sim$50 at $L = 12$), and this within-loop variation is what a model must learn. (b) At $L = 12$ the (expensive) Hepp bound is almost a deterministic function of the period, as observed by Balduf and Shaban (2024a).

Graph representation. Each row gives the *completion* of a Feynman graph: the graph obtained by joining the four external legs to one new vertex. Completions are simple 4-regular graphs with $L + 2$ vertices and $2(L + 2)$ edges, and all decompletions of a completion share the same period, so the completion is the natural (and non-redundant) input. We parse the edge-list column into an undirected `edge_index`, assert 4-regularity, and use $y = \log \mathcal{P}(G)$ as the regression target, since periods span several orders of magnitude (Fig. 1a) and errors in $\log \mathcal{P}$ are relative errors in $\mathcal{P}$. For the milestone we use $3 \leq L \leq 12$ (Table 1); $L = 12$ alone contributes 81,305 graphs of 14 vertices and 28 edges each. The complete $L = 13$ file (94 MB compressed) is the next step.

Table 1: Graphs per loop order used in the milestone (all primitive ϕ^4 completions).

L	3	4	5	6	7	8	9	10	11	12	total
graphs	1	1	2	5	14	49	227	1,354	9,722	81,305	92,680

Leakage-free splits. Many distinct graphs have *provably identical* periods (product, twist, Fourier and planar-duality symmetries), and the dataset stores them with bit-identical values. A random split would therefore place a graph in the test set whose exact label appears in training. We instead split *groups* of identical period values 80/10/10 into train/validation/test within every loop order, giving 74,202 / 9,145 / 9,333 graphs. The test set is dominated by $L = 11, 12$, where the problem is hardest.

4 Model Design and Metrics

Architectures. All GNNs share one skeleton so that only the message-passing operator varies: a linear node encoder, $K = 4$ message-passing layers of width 64 (each followed by BatchNorm, ReLU and a residual connection), a readout concatenating sum- and mean-pooling, and a two-layer MLP head. The operators are GCN (Kipf and Welling, 2017), $H^{(l+1)} = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)})$;

GraphSAGE (Hamilton et al., 2017), $h_v^{(l+1)} = \sigma(W^{(l)}[h_v^{(l)} \,\|\, \mathrm{mean}_{u\in N(v)} h_u^{(l)}])$; and GIN (Xu et al., 2019), $h_v^{(l+1)} = \mathrm{MLP}^{(l)}((1+\epsilon^{(l)})h_v^{(l)} + \sum_{u\in N(v)} h_u^{(l)})$. Models are trained with AdamW (lr $2 \cdot 10^{-3}$, cosine schedule, batch 256, 40 epochs) on the MSE of the standardized target; the checkpoint with the lowest validation MAE is evaluated on test. For constant features we drop BatchNorm (`-norm none`): with identical inputs every node embedding is identical, BatchNorm's running variance collapses to zero and evaluation-mode outputs diverge, which is what happened in our first attempt. Each run takes ≈5 minutes on one CPU core of an M1 Max laptop.

Why node features matter here. Every completed ϕ^4 graph is 4-regular. With constant input features, 1-WL colour refinement assigns every vertex of every such graph the same colour at every iteration, and message-passing GNNs are at most as expressive as 1-WL (Xu et al., 2019). A GNN with constant features can therefore compute nothing but a function of the number of vertices, i.e. of L: it can do no better than the per-loop mean, which is exactly the linear-regression baseline that earlier NNs failed to beat. This makes the input features, not the architecture, the crux of the problem, and motivates our ablation:

- **constant**: $x_v = 1$ (structure-free control);
- **random label**: one-hot of a uniformly random vertex permutation, resampled at every forward pass, i.e. genuinely arbitrary vertex labels as used by earlier GNNs;
- **canonical label**: one-hot of the vertex label *as stored in the dataset*. These labels are nauty's canonical form, so together with the edges they identify the graph up to isomorphism. This is not equivariant, but it is a strong structural signal rather than noise (see Section 5);
- **random (RNI)**: 8 i.i.d. Gaussian features resampled every forward pass (Abboud et al., 2021, Sato et al., 2021);
- **cycle participation** (ours): $c_k(v) = \#\{\text{simple cycles of length } k \text{ through } v\}$ for $k = 3, \ldots, 8$, as $\log(1 + c_k)$. We enumerate cycles exactly with a Numba-compiled DFS; summing $c_k(v)/k$ over v reproduces the dataset's graph-level cycle counts exactly, which we use as a correctness check;
- **LapPE** (ours): the 8 lowest non-trivial eigenvectors of the normalized Laplacian (Dwivedi et al., 2023), with random sign flips during training;
- **RWSE**: random-walk return probabilities $(P^t)_{vv}$, $t = 1..12$ (Dwivedi et al., 2022), a cheap proxy that mixes cycles with backtracking walks;
- combinations **cycle+LapPE** and **cycle+LapPE+RWSE**.

All structural features are computed in ≈45 s for the whole dataset on one core, many orders of magnitude cheaper than computing the periods themselves.

Non-GNN baselines. (i) Linear regression of $\log \mathcal{P}$ on L; (ii) ridge regression on one-hot(L) plus $\log(1 + \text{graph-level cycle counts})$, which uses the *same* information as our cycle features but pools it before learning, isolating the value of message passing; (iii) ridge on one-hot(L) plus log Hepp bound, reported as a reference only, because the Hepp bound is itself an expensive combinatorial quantity (a sum over all edge orderings).

Table 2: Test results on the grouped split (9,333 test graphs, mostly $L = 11, 12$). MRE = mean relative error of $\mathcal{P}$ in % (mean ± std over 2 seeds for GNNs); R^2_{w} = within-loop R^2 of $\log \mathcal{P}$. †Uses the Hepp bound, an expensive input, so it is a reference rather than a competitor. Constant-feature GNNs are trained without BatchNorm.

Non-GNN baseline	MRE (%)	R^2_{w}
Linear regression on L	45.60	-0.0018
Ridge on L + graph-level cycle counts	7.29	0.9654
Ridge on L + log Hepp bound†	0.60	0.9998
Ridge on L + cycles + Hepp†	0.41	0.9999

	GCN		GraphSAGE		GIN	
Node features	MRE (%)	R^2_{w}	MRE (%)	R^2_{w}	MRE (%)	R^2_{w}
constant	42.26 ± 0.30	-0.0307	42.35 ± 0.80	-0.0264	42.59 ± 0.93	-0.0240
random label	4.93 ± 0.20	0.9860	3.39 ± 0.09	0.9936	3.20 ± 0.31	0.9941
random (RNI)	30.00 ± 0.20	0.4832	26.39 ± 0.06	0.6044	27.31 ± 0.63	0.5727
canonical label	3.17 ± 0.06	0.9937	2.34 ± 0.04	0.9967	2.32 ± 0.18	0.9967
RWSE	2.87 ± 0.22	0.9950	1.31 ± 0.08	0.9989	2.00 ± 0.04	0.9974
LapPE	2.09 ± 0.07	0.9970	1.61 ± 0.06	0.9985	1.59 ± 0.04	0.9985
cycle	3.08 ± 0.00	0.9942	1.22 ± 0.08	0.9989	1.82 ± 0.07	0.9972
cycle+LapPE	2.06 ± 0.08	0.9972	1.41 ± 0.06	0.9986	1.43 ± 0.05	0.9987
cycle+LapPE+RWSE	1.22 ± 0.11	0.9991	0.97 ± 0.05	0.9994	1.02 ± 0.02	0.9993

Metrics. We report (a) the **mean relative error** $\frac{1}{N}\sum|\hat{\mathcal{P}} - \mathcal{P}|/\mathcal{P}$ (the metric of Balduf and Shaban, 2024a); (b) the MAE of $\log \mathcal{P}$; and (c) the **within-loop** R^2, $1 - \sum(\hat{y} - y)^2 / \sum(y - \bar{y}_L)^2$, where $\bar{y}_L$ is the mean true $\log \mathcal{P}$ of the graph's loop order. Ordinary R^2 is inflated by the trivial dependence on L; within-loop R^2 measures how much of the genuinely hard, within-L variance is explained (0 for the linear-in-L baseline).

5 Preliminary Results

Table 2 and Figure 2 summarize all runs; the main findings follow.

1. Structure-free GNNs cannot beat linear regression, as theory predicts. With constant node features all three architectures reach 42.3–42.6% MRE and a within-loop R^2 of -0.03. That is the per-loop-mean predictor, exactly as the 1-WL argument of Section 4 requires: no amount of message passing extracts structure from a regular graph with featureless nodes. Gaussian random features (RNI) break the symmetry but only reach 26–30% MRE. *Random one-hot labels* (a fresh random permutation at every forward pass) do much better, at 3.0–4.9%. Our interpretation is that one-hot labels from a small alphabet act as unique node identifiers: a node can recognize its own label returning after k hops and thus implicitly count cycles, which continuous Gaussian noise makes much harder to learn. Arbitrary labels are therefore not hopeless, but they need test-time randomness, are not invariant, and remain about 3× worse than explicit structural features.

2. Structural node features are what make GNNs work. Every structural encoding gives 1–3% MRE and within-loop $R^2 > 0.994$. The best configuration, cycle participation + LapPE +

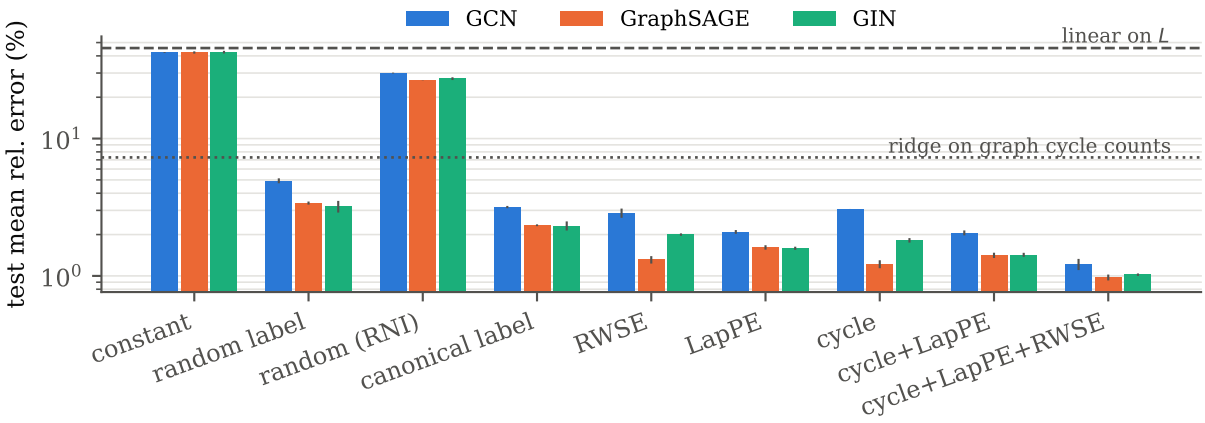


Figure 2: Node-feature ablation (log scale; error bars are ±1 std over 2 seeds). Dashed and dotted lines show the non-GNN baselines. Constant features cannot beat the linear-in-L baseline and Gaussian random features barely do; random one-hot labels already help (3–5%), and every explicit structural encoding beats ridge regression on the same cycle information by 2.4–7.5×.

RWSE, reaches $\mathbf{0.97 \pm 0.05\%}$ MRE with GraphSAGE and 1.02% with GIN, which is 47× better than linear regression on L and 7.5× better than ridge regression on the *same* cycle counts pooled to graph level (7.3%). Message passing over per-node cycle participation therefore extracts far more than the graph-level counts that earlier work used (Fig. 3). The three encodings are complementary: each alone gives 1.2–3.1%, and combining them is best for every architecture. Training is stable: the seed-to-seed std is at most 0.2 percentage points.

3. The dataset's vertex labels are not arbitrary. One-hot vertex labels taken directly from the files reach 2.2–3.2% MRE, close to the structural features. On inspection this is expected: the files store graphs in nauty's canonical labelling, so labels plus edges determine the isomorphism class, and the GNN can exploit the canonical ordering. Such features are not permutation equivariant and would break for graphs from another source. We therefore treat random-permutation labels as the true "arbitrary label" control, and consider this finding a caution for any work on this dataset.

4. Architecture matters less than features. With good features GraphSAGE ≈ GIN > GCN. GCN is consistently the weakest, plausibly because its symmetric-normalized aggregation mixes a node's own features with its neighbors' instead of keeping them separate as GraphSAGE's concatenation and GIN's $(1+\epsilon)$ term do. GIN's extra theoretical expressivity brings no gain here, which is consistent with point 1: on regular graphs the bottleneck is the input features, not the injectivity of the aggregator.

5. Gap to the Hepp bound, and extrapolation in L. The expensive Hepp bound alone gives 0.6% MRE in a ridge model, so our best cheap GNN is not yet as accurate as a predictor built on this feature (it is ∼1.6× worse), although it needs only per-node cycle counts and spectra. For comparison, label noise is negligible (median relative uncertainty of the $L = 12$ periods is 6×10^{-5}). Table 3 reports a harder and practically more relevant setting: training only on $L \leq 11$ (about 11k graphs) and predicting all 81,305 graphs at $L = 12$. Out of the box, every GNN fails to extrapolate

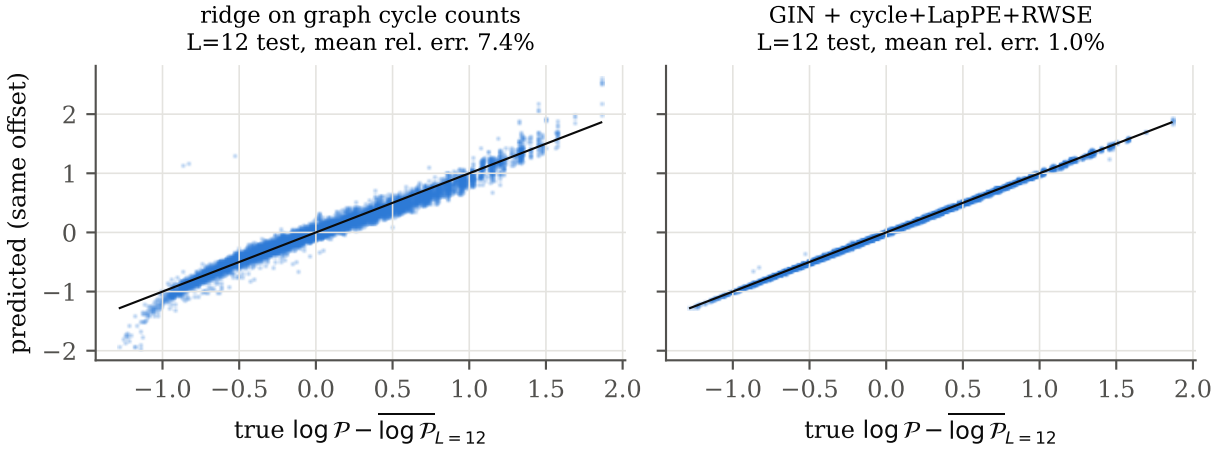


Figure 3: Predicted versus true $\log \mathcal{P}$ for the $L = 12$ test graphs, after subtracting the mean $\log \mathcal{P}$ at $L = 12$ so that only the hard, within-loop variation is shown. Left: ridge regression on graph-level cycle counts. Right: the best GNN by validation error.

(30–58% MRE, often worse than linear regression), while ridge on cycle counts degrades more gracefully (19.7%). The diagnosis is that the networks must extrapolate the large, trivial growth of $\log \mathcal{P}$ with L through sum-pooled embeddings of a graph size they have never seen. A one-line change fixes this: subtracting a linear fit of $\log \mathcal{P}$ on L and training the GNN on the residual (`-resid_L`). With it, GraphSAGE + cycle+LapPE+RWSE predicts the unseen $L = 12$ periods with **4.3%** MRE (within-loop $R^2 = 0.991$), 4.6× better than ridge regression, using only ∼10k training graphs. This is the most encouraging result for the real use case of predicting high-loop periods from cheaper low-loop data.

What did not work / lessons. (i) With BatchNorm, constant-feature models diverged at evaluation time (MRE $\to \infty$). The cause is degenerate batch statistics, and we re-ran these controls without normalization. (ii) Our first attempt stored the label feature under a name containing `index`, which PyG silently treats as an index tensor and offsets during batching. (iii) Hyperparameters (width 64, depth 4, 40 epochs) are untuned. Validation error is noisy at the initial learning rate and drops as the cosine schedule anneals, and for every run the best checkpoint lies within the last five epochs (Fig. 4), so longer training will likely improve all GNN numbers.

6 Next Steps

- **More data and larger L.** Add the complete $L = 13$ set (94 MB compressed) and the $14 \le L \le 18$ samples. Because those samples are non-uniform, we will evaluate them per loop order and report how error grows with L, which is the regime where cheap predictors matter most.
- **Extrapolation.** Make training on low L and testing on high L a primary benchmark (train ≤ 12, test 13; train ≤ 13, test 14–18), and study which features stay calibrated across sizes (cycle counts grow with L and may need per-L normalization).
- **Richer and more principled structural features.** Use longer cycle lengths and per-edge cycle counts as edge features, sign- and basis-invariant Laplacian encodings (e.g. SignNet),

Table 3: Extrapolation in loop order: train/validate on $L \leq 11$, test on all 81,305 $L = 12$ graphs (single seed, 80 epochs). Baselines use L as a numeric feature so that they can extrapolate.

Model	MRE (%)	R^2_{w}
Linear regression on L	48.3	-0.0067
Ridge on L + graph-level cycle counts	19.7	0.8465
Ridge on L + log Hepp bound†	0.65	0.9997
GCN + canonical label	41.0	-0.589
GraphSAGE + canonical label	57.6	-1.937
GIN + canonical label	36.0	-0.078
GCN + RWSE	30.5	0.702
GraphSAGE + RWSE	9.6	0.937
GIN + RWSE	41.0	-0.165
GCN + cycle	52.7	-1.087
GraphSAGE + cycle	55.8	-1.391
GIN + cycle	53.5	-1.221
GCN + cycle+LapPE+RWSE	44.2	-0.310
GraphSAGE + cycle+LapPE+RWSE	34.2	0.314
GIN + cycle+LapPE+RWSE	29.6	0.511
GNN predicts residual of linear fit on L		
GCN + RWSE	43.3	0.368
GraphSAGE + RWSE	10.9	0.952
GIN + RWSE	32.3	0.667
GCN + cycle+LapPE+RWSE	7.0	0.973
GraphSAGE + cycle+LapPE+RWSE	4.3	0.991
GIN + cycle+LapPE+RWSE	5.2	0.985

and subgraph-count features motivated by graph-theoretic invariants of periods (vertex cuts, Martin invariants). We will also compare a pure "set of node features" model (DeepSets, no message passing) to isolate what message passing adds.

- **Ablations and tuning.** Sweep depth, width and pooling, train longer, use 5 seeds, and examine per-length cycle-feature importance (dropping one cycle length at a time) as an explainability analysis of which cycles determine the period.
- **Cost.** Measure end-to-end wall-clock time per graph for our features + GNN against the Hepp bound and against numerical integration, so that the accuracy/speed trade-off is explicit.
- **Deliverables.** Port the pipeline (already modular and documented in `code/`) into a single Colab notebook and write the Medium blog post with these figures and PyG code snippets.

Code. All code for this milestone is in `code/`: `data.py` (download format, parsing, features, grouped splits), `models.py`, `train.py`, `baselines.py`, `make_figures.py`, and shell scripts that reproduce every number in this report.

References

R. Abboud, İ. İ. Ceylan, M. Grohe, and T. Lukasiewicz. The surprising power of graph neural networks with random node initialization. In *IJCAI*, 2021.

P.-H. Balduf. Statistics of Feynman amplitudes in ϕ^4-theory. *JHEP*, 11:160, 2023. arXiv:2305.13506.

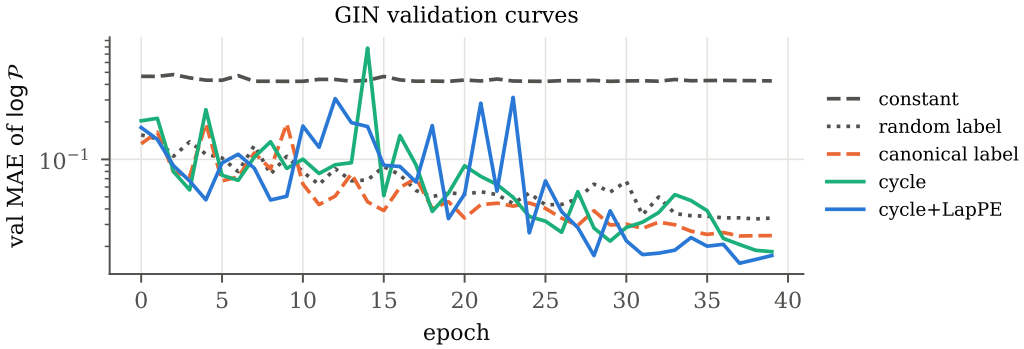


Figure 4: Validation MAE of $\log \mathcal{P}$ during training (GIN, seed 0). Curves are noisy at the initial learning rate and settle as the cosine schedule anneals; the constant-feature control stays flat at the per-loop-mean level throughout.

P.-H. Balduf and K. Shaban. Predicting Feynman periods in ϕ^4-theory. arXiv:2403.16217, 2024.

P.-H. Balduf and K. Shaban. Feynman periods in ϕ^4-theory [dataset]. Borealis Dataverse, 2024. doi:10.5683/SP3/NLEDGH.

V. P. Dwivedi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson. Graph neural networks with learnable structural and positional representations. In *ICLR*, 2022.

V. P. Dwivedi, C. K. Joshi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson. Benchmarking graph neural networks. *JMLR*, 24(43):1–48, 2023.

W. L. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *NeurIPS*, 2017.

T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

M. V. Kompaniets and E. Panzer. Minimally subtracted six-loop renormalization of $O(n)$-symmetric ϕ^4 theory and critical exponents. *Phys. Rev. D*, 96:036016, 2017. arXiv:1705.06483.

R. Sato, M. Yamada, and H. Kashima. Random features strengthen graph neural networks. In *SDM*, 2021.

K. G. Wilson and J. Kogut. The renormalization group and the ϵ expansion. *Physics Reports*, 12(2):75–199, 1974.

K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? In *ICLR*, 2019.